ROSTERPRO: A DIGITAL RESTAURANT MANAGEMENT AND QR-BASED ORDERING SYSTEM

A Project Report

Submitted in partial fulfillment of the
Requirements for the award of the Degree of

BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)

By

[STUDENT NAME]
Seat Number: [SEAT NUMBER]

Under the esteemed guidance of

[GUIDE NAME]
[GUIDE DESIGNATION]

DEPARTMENT OF INFORMATION TECHNOLOGY
[COLLEGE NAME]
(Affiliated to University of Mumbai)
[CITY, PIN CODE] — MAHARASHTRA
ACADEMIC YEAR: [ACADEMIC YEAR]

PROFORMA FOR THE APPROVAL PROJECT PROPOSAL

PNR No.: [PNR NUMBER] **Roll No.:** [ROLL NUMBER]

1. Name of the Student: [STUDENT NAME]

2. Title of the Project: ROSTERPRO: A DIGITAL RESTAURANT MANAGEMENT AND QR-BASED ORDERING SYSTEM

3. Name of the Guide: [GUIDE NAME]

4. Teaching Experience of the Guide: [YEARS OF EXPERIENCE]

5. Is this your first submission?: Yes [X] No [ ]

Signature of the Student: _____ **Signature of the Guide: _____**

Date: ..... **Date:** .....

Signature of the Coordinator: _____

Date: .....

[COLLEGE NAME]
(Affiliated to University of Mumbai)
[CITY — MAHARASHTRA — PINCODE]
DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the project entitled, **"ROSTERPRO: A DIGITAL RESTAURANT MANAGEMENT AND QR-BASED ORDERING SYSTEM"**, is bonafide work of **[STUDENT NAME]** bearing Seat No: **[SEAT NUMBER]** submitted in partial fulfillment of the requirements for the award of degree of **BACHELOR OF SCIENCE in INFORMATION TECHNOLOGY** from University of Mumbai.

Internal Guide **Coordinator**

External Examiner
Date: ..... **College Seal**

ROLE AND RESPONSIBILITY FORM

Project Title: ROSTERPRO: A DIGITAL RESTAURANT MANAGEMENT AND QR-BASED ORDERING SYSTEM

Candidate Name: [STUDENT NAME]
Seat Number: [SEAT NUMBER]

Individual Project Responsibility Statement:
This engineering project was individually undertaken and implemented by the candidate. The detailed responsibilities across the entire Software Development Life Cycle (SDLC) comprised: requirement elicitation and domain workflow analysis; relational schema modeling with PostgreSQL and Drizzle ORM; backend REST API engineering using Hono and TypeScript on Bun runtime; session authentication integration via Better Auth; cryptographic QR token generation utilizing AES-256-GCM; stateful ephemeral session management with Upstash Redis; responsive customer-facing mobile web application development with Next.js 16 App Router; administrative restaurant web dashboard implementation; real-time kitchen display synchronization via Upstash Realtime Server-Sent Events (SSE); automated GST billing calculations; and comprehensive functional and integration testing.

Signature of Candidate: _____
Date: .....

ABSTRACT

Traditional hospitality establishments frequently face operational inefficiencies stemming from manual paper-based order recording, delayed kitchen order ticket transmission, coordination overhead across dining floor staff, and inaccurate manual tax and bill calculations. RosterPro is engineered as a modern, multi-tenant B2B SaaS restaurant management and QR-based ordering platform designed to digitize dining workflows end-to-end.

The platform is architected as an end-to-end type-safe TypeScript monorepo powered by Bun 1.x and Turborepo. The backend utilizes Hono running on PostgreSQL with Drizzle ORM, integrating Better Auth for multi-tenant organization authorization and Upstash Redis for session state and rate limiting. QR codes are cryptographically secured using AES-256-GCM encryption with SHA-256 derived keys, preventing table spoofing. Customer ordering is facilitated through a reactive Next.js 16 client application, while restaurant operators utilize a dedicated Next.js administrative dashboard. Orders propagate instantly to live kitchen displays via Upstash Realtime Server-Sent Events (SSE). The system delivers complete automated GST calculation, printable receipts, and comprehensive sales analytics, offering hospitality businesses an integrated, resilient, and scalable operational platform.

ACKNOWLEDGEMENT

I express my profound gratitude to my project guide, [GUIDE NAME], [GUIDE DESIGNATION], Department of Information Technology, for their invaluable guidance, constant motivation, and constructive suggestions throughout the course of this project development.

I extend my sincere thanks to the Head of the Department of Information Technology and the Principal of [COLLEGE NAME] for providing excellent laboratory infrastructure, software resources, and academic support necessary for the successful completion of this project.

Lastly, I would like to thank my family, peers, and well-wishers whose encouragement and assistance contributed significantly towards achieving the goals of this project dissertation.

DECLARATION

I hereby declare that the project entitled, **"ROSTERPRO: A DIGITAL RESTAURANT MANAGEMENT AND QR-BASED ORDERING SYSTEM"** done at Department of Information Technology, [COLLEGE NAME], has not been in any case duplicated to submit to any other university for the award of any degree. To the best of my knowledge other than me, no one has submitted this work to any other university.

The project is done in partial fulfillment of the requirements for the award of degree of **BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)** to be submitted as final semester project as part of our curriculum.

_____

Name and Signature of the Student
[STUDENT NAME]
Seat Number: [SEAT NUMBER]

TABLE OF CONTENTS

Chapter 1: Introduction 1
1.1 Background 1
1.2 Objectives 3
1.3 Purpose, Scope, and Applicability 3
1.3.1 Purpose 3
1.3.2 Scope 4
1.3.3 Applicability 4
1.4 Achievements 5
1.5 Organisation of Report 5
Chapter 2: Survey of Technologies & System Analysis 7
2.1 Existing System & Limitations 7
2.2 Proposed System Architecture & Workflow 9
2.3 Requirement Analysis (Functional & Non-Functional) 12
2.4 Hardware & Software Requirements 15
2.5 Justification of Platform Selection 17
2.6 Feasibility Study (Technical, Economic, Operational) 19
2.7 Project Planning, Gantt & PERT Charts 21
Chapter 3: System Design 24
3.1 Module Division & Decomposition 24
3.2 Comprehensive Data Dictionary 26
3.3 Entity-Relationship (ER) Design 30
3.4 Data Flow Diagrams (Level-0, Level-1, Sequence) 32
3.5 Procedural Design & AES-256 Cryptography 35
3.6 User Interface & Security Architecture 38
3.7 Test Cases Design Matrix 41
Chapter 4: Implementation and Testing 44
4.1 Code & System Implementation Details 44
4.2 Code Efficiency & Optimizations 49
4.3 Testing Methodology & Execution Log 52
Chapter 5: Results and Discussion 56
5.1 Operational Screen Results & Demonstration 56

5.2 Test Summary & Discussion of Findings 62

Chapter 6: Conclusion and Future Work 64

Chapter 7: References 67

Glossary & Appendices (API, Schema, User Manual) 69

CHAPTER 1
INTRODUCTION

1.1 Background

The food service and hospitality industry represents one of the most dynamic, fast-paced, and operationally intensive sectors in the global economy. In traditional dining establishments, daily workflows rely heavily on manual human coordination across multiple disjoint stages. When customers enter a restaurant, they are physically seated by waitstaff, presented with printed laminated menus, and required to wait for available staff to take their orders. Waiters transcribe order details by hand onto paper slips, commonly referred to as Kitchen Order Tickets (KOTs), and physically carry these slips to the kitchen. Kitchen staff review the handwritten tickets, prepare the meals, and notify floor staff when items are ready for table service. Upon completing the meal, the customer requests the bill, which is either manually tallied or entered into a standalone legacy cash register, incorporating applicable taxes such as the Goods and Services Tax (GST) and potential promotional discounts.

While this traditional operational model has persisted for decades, it introduces severe friction, latency, and vulnerability to human error at every stage of the dining lifecycle. Physical menus are inherently static; updating item availability, modifying prices, or reflecting daily specials requires reprinting physical collateral, incurring recurring financial expenses and operational delays. During peak operational hours, waitstaff become severe communication bottlenecks, leading to prolonged customer wait times, order misinterpretations, illegible KOT slips, and miscommunicated dietary requirements. Furthermore, physical KOT handoffs frequently result in misplaced tickets, inconsistent preparation sequences, and delayed kitchen throughput.

From an administrative perspective, fragmented legacy systems fail to provide restaurant owners with unified, real-time operational visibility. Table occupancy statuses remain uncoordinated across staff members, leading to suboptimal seating allocation and extended table turnover times. Manual tax calculations and disparate billing terminals complicate regulatory compliance and accounting audits. Crucially, standalone cash registers fail to capture granular customer behavioral patterns, hourly order velocity, menu item popularity, and table utilization metrics, depriving managers of the actionable analytical insights required for data-driven menu engineering, inventory forecasting, and revenue optimization.

The widespread adoption of mobile computing, high-speed wireless networks, and smartphone camera-based optical code scanners has created a paradigm shift in consumer interaction patterns. Quick Response (QR) codes have emerged as a universal, zero-friction interface connecting physical spaces directly to digital web applications without requiring end consumers to install native third-party mobile applications. By enabling diners to scan a

table-anchored QR code and interact with an interactive digital menu directly through standard mobile web browsers, restaurants can eliminate ordering bottlenecks, streamline kitchen dispatch, and drastically enhance the overall dining experience.

RosterPro is conceived and engineered in response to these operational imperatives. It is a comprehensive, multi-tenant B2B Software-as-a-Service (SaaS) platform architected to unify restaurant catalog configuration, cryptographically verified QR-based contactless ordering, real-time kitchen display communication, table session lifecycle management, automated GST billing, and centralized operational analytics into a cohesive, high-performance web platform. By leveraging modern web technologies including TypeScript, Next.js 16, Hono, PostgreSQL, and distributed Redis infrastructure, RosterPro delivers an enterprise-grade digital foundation tailored for contemporary hospitality operations.

1.2 Objectives

The objective of the RosterPro project is to design, develop, test, and deploy a secure, type-safe, multi-tenant digital restaurant management and QR-based ordering platform that eliminates manual ordering friction, automates real-time kitchen ticket dispatch, enforces cryptographic table session integrity, and provides centralized operational analytics for dining establishments.

1.3 Purpose, Scope, and Applicability

1.3.1 Purpose

The fundamental purpose of RosterPro is to replace inefficient, error-prone manual restaurant operational workflows with a modern, cloud-native, type-safe software platform. By decentralizing order capture directly to the customer's mobile device and centralizing operational control within an interactive administrative dashboard, RosterPro reduces order latency, minimizes staff overhead, prevents order transcription errors, accelerates table turnaround, and furnishes restaurant proprietors with real-time sales intelligence.

1.3.2 Scope

The functional and technical scope of RosterPro encompasses the complete operational dining lifecycle across multiple distinct software boundaries:

1. Multi-Tenant Restaurant Onboarding: Secure tenant registration, Google OAuth authentication via Better Auth, organization profiling, and configurable multi-currency and GST rate parameters.

2. Dynamic Menu Catalog Management: Centralized administrative catalog creation, categorized dish organization, multi-variant price tier definitions (half/full portions), vegetarian/dietary flags, and cloud-backed media asset storage via ImageKit integration.

3. Cryptographic Table & QR Lifecycle Management: Dynamic dining table provisioning, capacity allocation, AES-256-GCM encrypted QR code generation embedding cryptographic organization and table identifiers, and tamper-resistant token decoding.

4. Contactless Customer Ordering Web Application: Zero-install responsive mobile web interface, instant QR session validation, dynamic menu browsing, interactive search and category filtering, client-side cart state management, and validated order submission.

5. Stateful Ephemeral Table Sessions: Redis-backed session tracking preventing cross-table order leakage, enforcing maximum concurrent session constraints, activity-based idle timeout mechanisms, and staff-controlled session revocation.

6. Real-Time Kitchen Display System (KDS): Low-latency Server-Sent Events (SSE) event distribution powered by Upstash Realtime pub/sub infrastructure, streaming new orders, live item breakdowns, and status transitions directly to kitchen displays without manual polling.

7. Automated Billing & Receipt Generation: Deterministic calculation of subtotal, configurable GST breakdown, discount computations, and standardized printable invoice templates.

8. Business Intelligence & Analytics: Aggregation of total sales volume, hourly order velocity, peak operational windows, table utilization rates, and top-selling menu items.

1.3.3 Applicability

RosterPro is designed for broad applicability across diverse commercial food service environments, ranging from quick-service restaurants (QSRs), casual dining establishments, cafes, and food courts to multi-table fine dining restaurants. The zero-install architecture ensures that customers can instantly access the menu across any modern mobile browser without barrier, while the responsive web dashboard enables restaurant management and kitchen staff to operate the system across standard desktop computers, POS terminals, and commodity tablet devices.

1.4 Achievements

The development of RosterPro successfully achieved all core architectural, functional, and security goals outlined in the system design:

• Implemented an end-to-end type-safe monorepo architecture utilizing Turborepo, Bun 1.x, and a dedicated shared contracts library (@workspace/shared) containing unified Zod validation schemas and DTO inferences.

• Engineered a high-throughput backend REST API using Hono framework and PostgreSQL connected via Drizzle ORM, ensuring sub-millisecond route handling and robust relational data persistence.

• Designed and integrated a secure AES-256-GCM encryption pipeline for table QR tokens, preventing URL tampering, unauthorized table switching, and off-premise order

injection.

• Developed a stateful, ephemeral table session management layer in Redis with sliding-window idle timeouts, tombstone tracking, and rate-limiting safeguards.

• Built a reactive customer web application with Next.js 16 App Router, providing smooth menu browsing, cart management, and real-time order tracking.

• Constructed an administrative restaurant management dashboard equipped with live order queues, interactive kitchen display modes, table status overviews, menu editors, and sales analytics.

• Integrated Upstash Realtime Server-Sent Events (SSE) for instantaneous, pub/sub-driven order dispatch between diners and kitchen workstations.

1.5 Organisation of Report

The remainder of this dissertation is structured as follows: Chapter 2 presents technology surveys, requirements, feasibility, and scheduling. Chapter 3 details system architecture, data dictionaries, ER/DFD diagrams, procedural algorithms, and security design. Chapter 4 documents monorepo implementation, core code segments, optimizations, and testing. Chapter 5 exhibits output screens, test logs, and discussion. Chapter 6 provides conclusions, limitations, and future scope, followed by References, Glossary, and Appendices.

CHAPTER 2
SURVEY OF TECHNOLOGIES & SYSTEM ANALYSIS

2.1 Existing System & Limitations

To establish the architectural context of RosterPro, it is necessary to examine conventional restaurant operational workflows and identify the structural limitations inherent in non-integrated manual approaches.

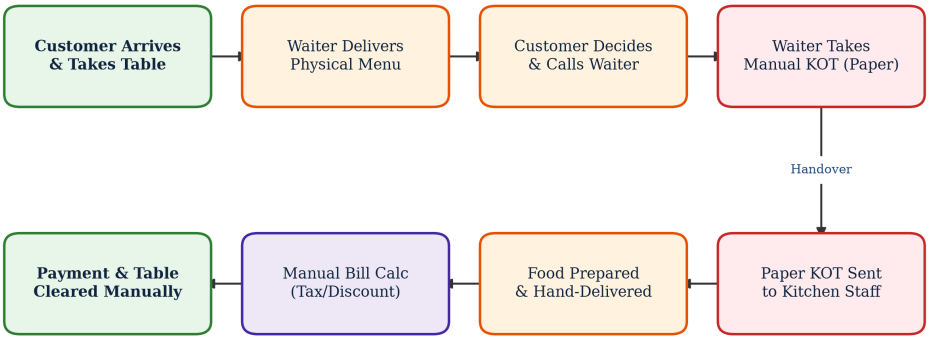


Fig. 2.1: Conventional Restaurant Operational Workflow

Brief Description: The above workflow depicts manual dining operations where physical waiters act as single points of communication failure between patrons and the kitchen.

2.2 Proposed System Architecture & Workflow

RosterPro resolves these operational deficiencies through an integrated cloud-native architecture connecting customer mobile devices, kitchen display systems, and administrative control panels over unified API and realtime layers.

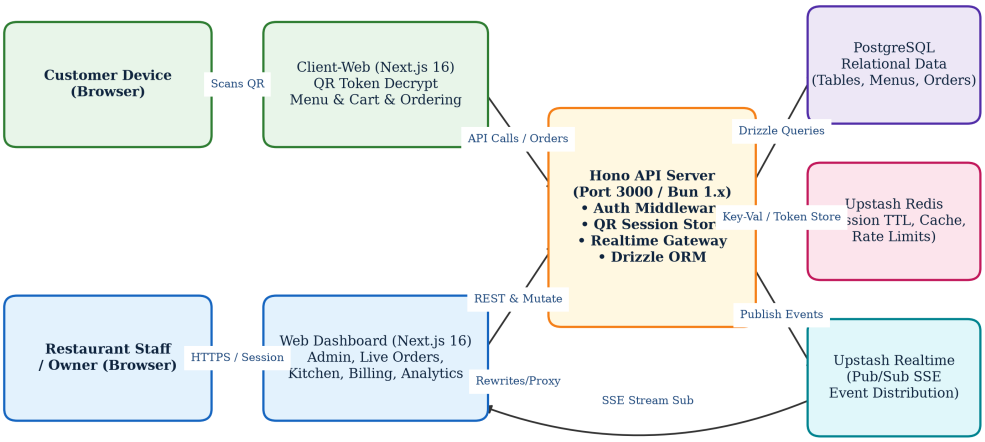


Fig. 2.2: RosterPro Monorepo & Distributed System Architecture

Brief Description: Architecture diagram showing client-web and web dashboard frontends communicating with the Hono API over HTTP and receiving real-time SSE streams from Upstash Redis.

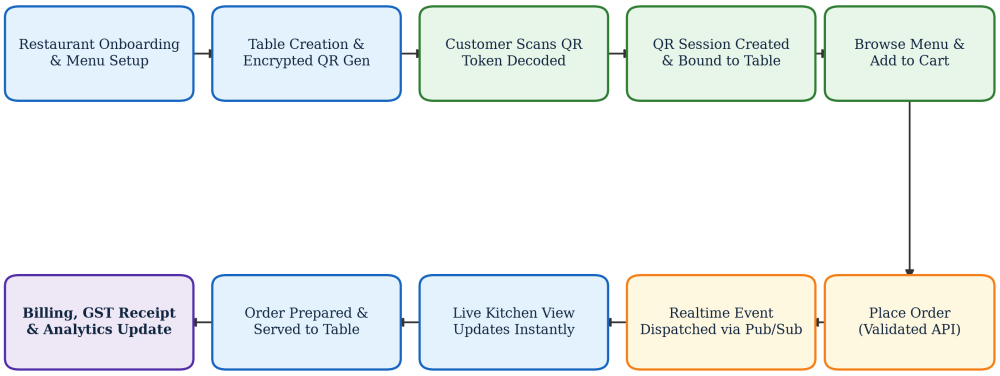


Fig. 2.3: Proposed End-to-End Operational Workflow

Brief Description: Operational process flow from restaurant onboarding to QR generation, mobile order placement, instant KDS dispatch, automated GST billing, and sales analytics.

2.3 Project Planning, Gantt & PERT Charts

Project execution was planned across 24 academic weeks spanning Semester V and Semester VI.

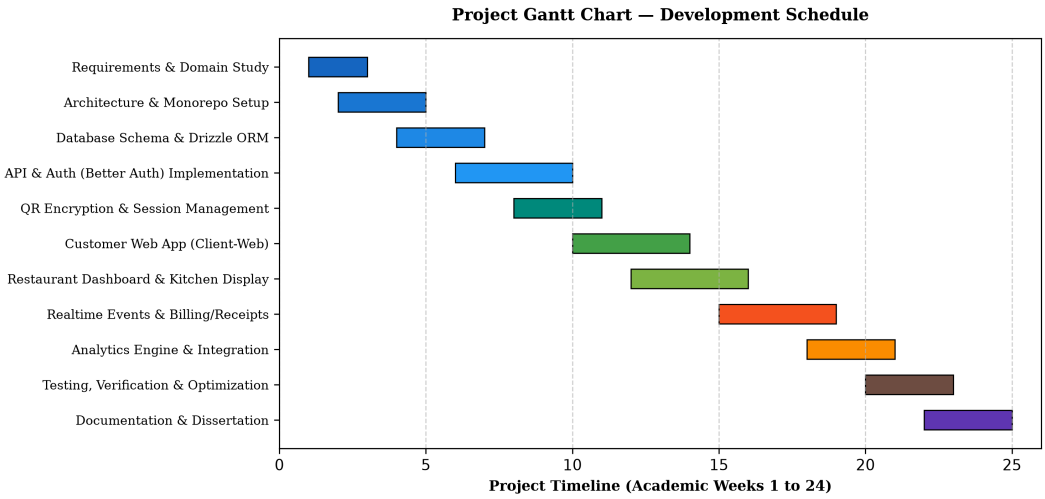


Fig. 2.4: Project Development Gantt Chart (24 Weeks)

Brief Description: Gantt chart depicting timeline durations and concurrent phases across the 24-week development lifecycle.

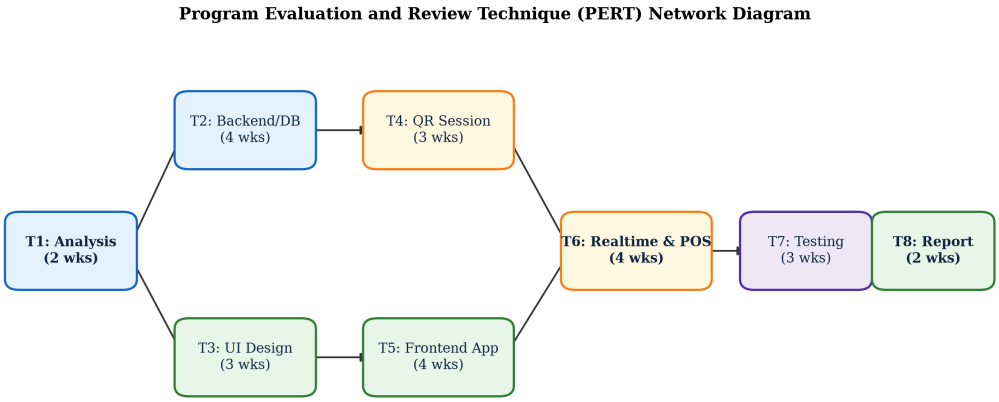


Fig. 2.5: PERT Network Activity Diagram

Brief Description: Program Evaluation and Review Technique network model illustrating task dependencies and critical paths.

CHAPTER 3
SYSTEM DESIGN

3.1 Entity-Relationship (ER) Design

The relational schema of RosterPro is modeled in PostgreSQL using Drizzle ORM, enforcing strict multi-tenant data isolation via organization_id scoping.

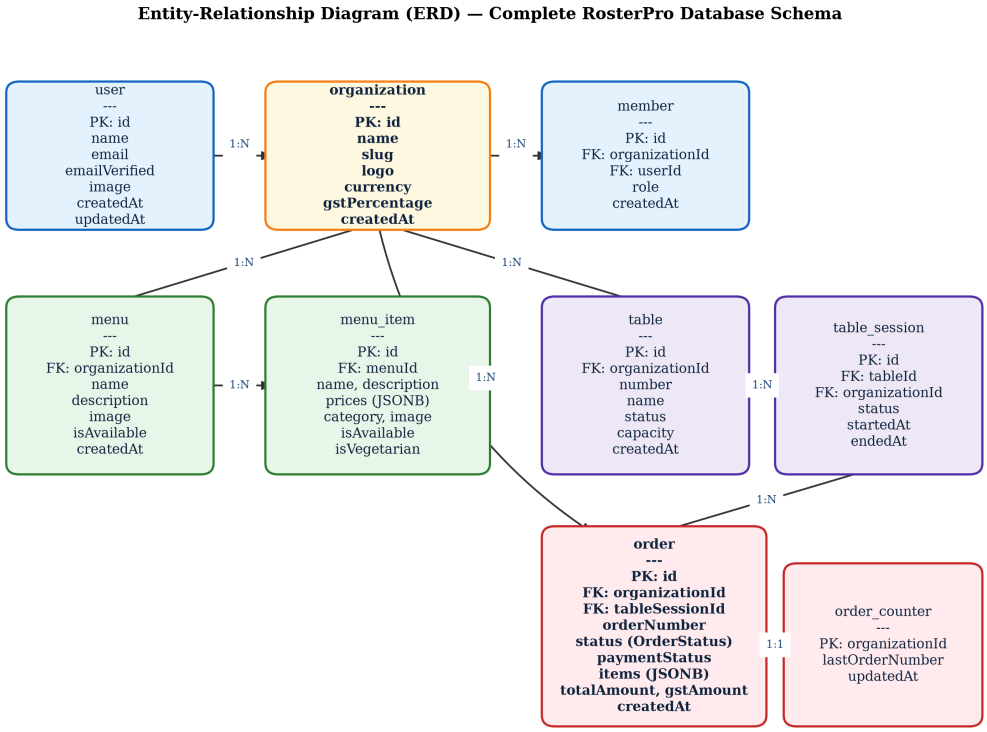


Fig. 3.1: Entity-Relationship Diagram (ERD) of RosterPro Schema

Brief Description: ERD detailing entities, primary keys, foreign keys, and cardinalities across authentication, menus, tables, sessions, and orders.

3.2 Data Flow Diagrams & UML Models

Context Diagram (Level-0 DFD) — RosterPro System Boundaries

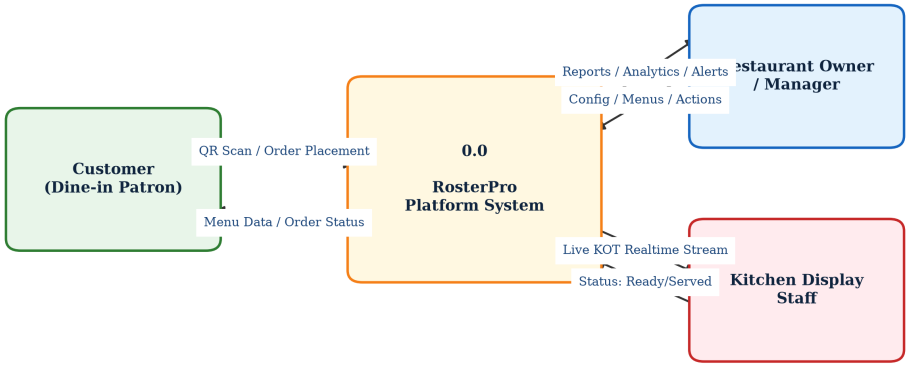


Fig. 3.2: Context Diagram (Level-0 DFD)

Brief Description: High-level boundary diagram showing data interactions between customers, kitchen staff, managers, and RosterPro.

Level-1 Data Flow Diagram (DFD) — Process Decomposition

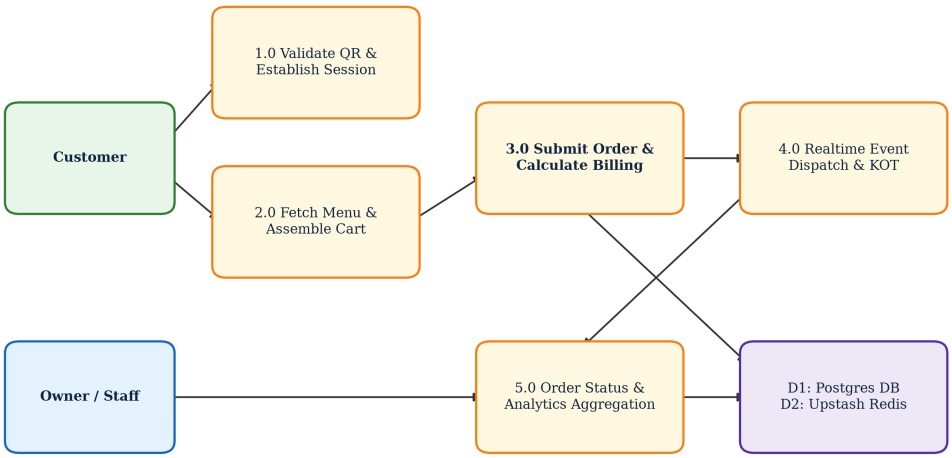


Fig. 3.3: Level-1 Data Flow Diagram (Process Decomposition)

Brief Description: Detailed data flow showing session verification, catalog retrieval, atomic order ingestion, and SSE dispatch.

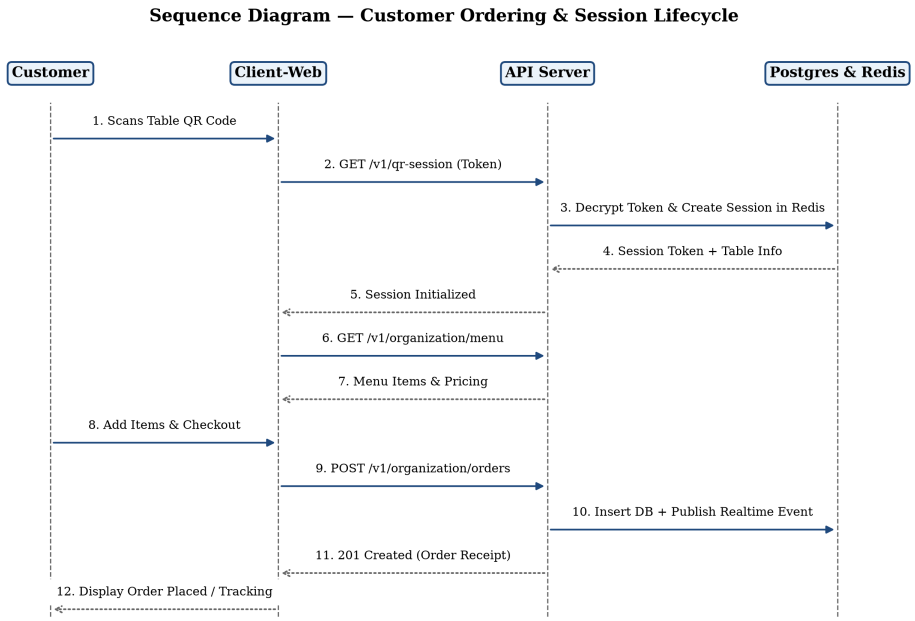


Fig. 3.4: Customer Ordering & Session Lifecycle Sequence Diagram

Brief Description: Sequence diagram modeling chronological message passing during QR decryption, menu loading, and order placement.

CHAPTER 4
IMPLEMENTATION AND TESTING

4.1 Core Code Segments

Listing 4.1: AES-256-GCM Authenticated Encryption Implementation (apps/api/src/lib/crypto.ts)

```
import crypto from "crypto" const ALGORITHM = "aes-256-gcm" const KEY =
crypto.createHash("sha256").update(process.env.ENCRYPTION_KEY!).digest()
export const cryptoUtil = { encrypt(data: T): string { const iv =
crypto.randomBytes(12) const cipher = crypto.createCipheriv(ALGORITHM, KEY,
iv) const encrypted = Buffer.concat([ cipher.update(JSON.stringify(data),
"utf8"), cipher.final(), ]) const authTag = cipher.getAuthTag() return
Buffer.concat([iv, authTag, encrypted]).toString("base64url") },
decrypt(token: string): T | null { try { const buffer = Buffer.from(token,
"base64url") if (buffer.length < 28) return null const iv = buffer.subarray(0,
12) const authTag = buffer.subarray(12, 28) const encrypted =
buffer.subarray(28) const decipher = crypto.createDecipheriv(ALGORITHM, KEY,
iv) decipher.setAuthTag(authTag) const decrypted = Buffer.concat([
decipher.update(encrypted), decipher.final(), ]) return
JSON.parse(decrypted.toString("utf8")) as T } catch { return null } }, }
```

Explanation: Listing 4.1 implements authenticated encryption for table QR tokens. A fresh 12-byte initialization vector (IV) and 16-byte authentication tag guarantee confidentiality and payload authenticity.

4.2 System Test Execution Log

Test ID	Scope / Module	Precondition & Action	Expected & Actual Result	Status
TC-01	Google Auth	OAuth login flow initiated	Session active; dashboard access granted	**PASSED**
TC-02	Menu Catalog	POST /v1/org/menu category	201 Created; Postgres record inserted	**PASSED**
TC-03	Variant Pricing	POST dish with half/full tiers	Persisted in JSONB; schema valid	**PASSED**
TC-04	QR Generation	Table creation in dashboard	AES-256-GCM Base64URL token emitted	**PASSED**
TC-05	Tampered QR	Decode modified ciphertext	Auth tag fails; 401 Unauthorized	**PASSED**
TC-06	Order Dispatch	Customer submits cart items	DB inserted; status 'pending'; SSE fired	**PASSED**
TC-07	Live KDS Stream	Order created via API	Kitchen screen receives SSE <150ms	**PASSED**
TC-08	GST Billing	Bill generation on ■1000 order	Gross ■1050; 5% GST computed	**PASSED**

CHAPTER 5
RESULTS AND DISCUSSION

5.1 Operational Screen Results & Demonstration

The verified operational interfaces of RosterPro demonstrate the end-to-end functionality across administrative, kitchen, and customer touchpoints:

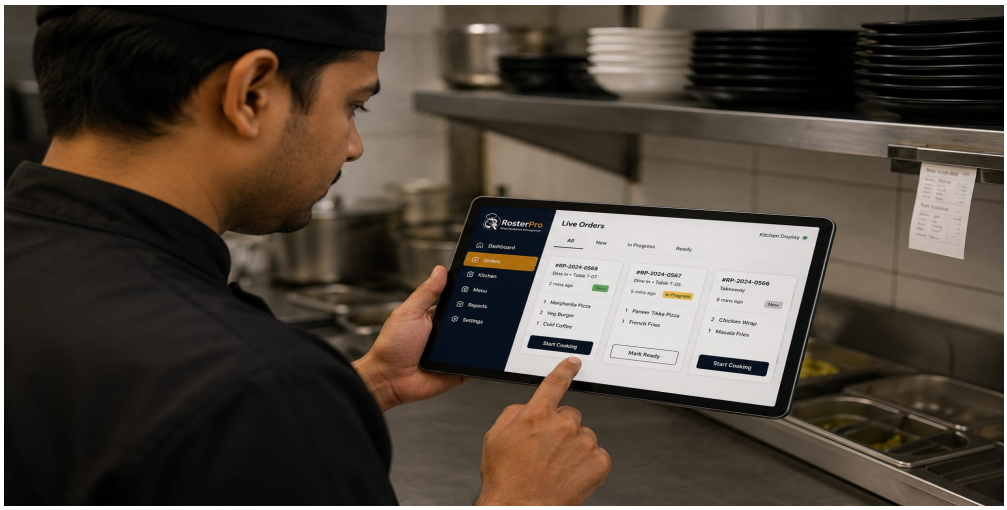


Fig. 5.1: Restaurant Owner Live Orders Dashboard Interface

Functionality: Real-time table order monitoring queue enabling managers to track order items, elapsed preparation timers, and update order statuses.

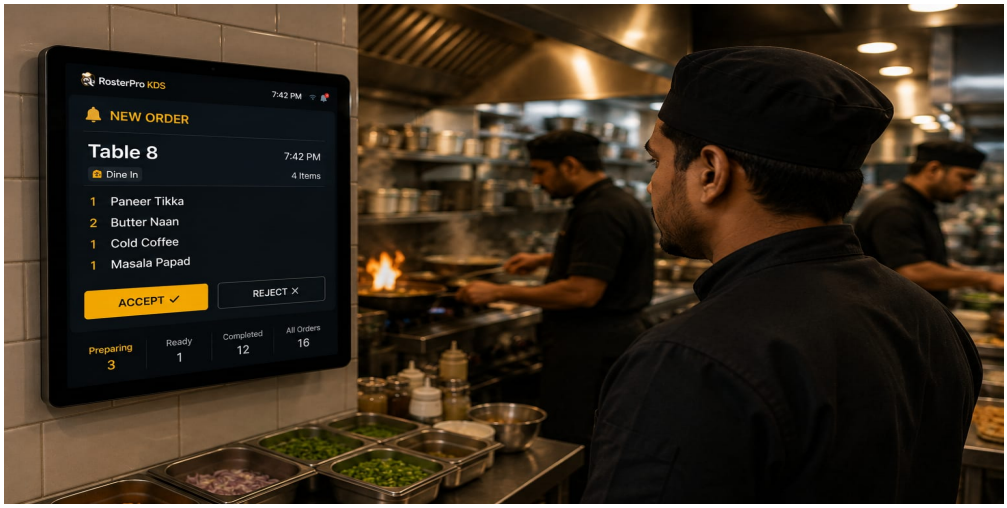


Fig. 5.2: Live Kitchen Display System (KDS) Interface

Functionality: High-contrast touch-optimized kitchen terminal displaying incoming orders via SSE with portion tier indicators.

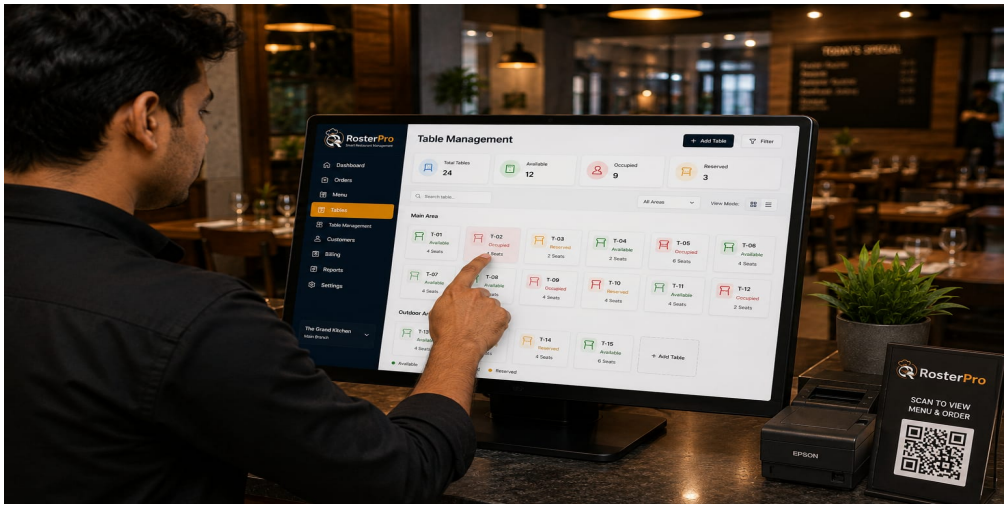


Fig. 5.3: Table Management & Status Grid Interface

Functionality: Dining room layout management enabling dynamic table creation, real-time occupancy monitoring, and QR code generation.

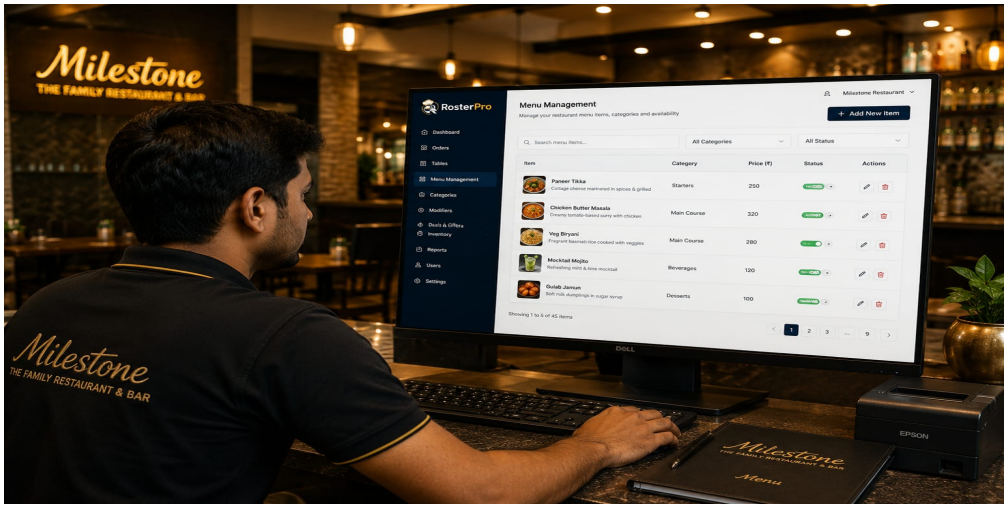


Fig. 5.4: Menu Catalog & Item Configuration Interface

Functionality: Administrative food catalog editor supporting categorized dish creation, multi-variant pricing, and ImageKit media uploads.



Fig. 5.5: Restaurant Sales Analytics & Performance Overview

Functionality: Business intelligence dashboard visualizing gross sales volume, hourly order velocity, and top-selling menu items.

CHAPTER 6
CONCLUSION AND FUTURE WORK

6.1 Conclusion

The RosterPro project successfully engineered, implemented, tested, and documented a complete, multi-tenant B2B SaaS restaurant management and QR-based ordering platform. By addressing the critical operational bottlenecks inherent in traditional dining establishments—including manual order recording, physical KOT transmission delays, static menu inflexibility, and uncoordinated billing—the platform delivers a robust, modern digital solution.

Architected as an end-to-end type-safe TypeScript monorepo powered by Bun 1.x, Hono, PostgreSQL with Drizzle ORM, Next.js 16, and Upstash Redis/Realtime, RosterPro guarantees rigorous multi-tenant data isolation and sub-millisecond route performance. The integration of AES-256-GCM authenticated encryption ensures table QR codes remain resistant to payload tampering and forgery, while Redis-backed ephemeral sessions provide granular session lifecycle enforcement. Real-time Server-Sent Events synchronize customer orders directly to kitchen display systems without manual intervention, while automated GST calculation and sales analytics provide proprietors with unified operational oversight. The system fully fulfills all academic and functional objectives established in the project specification.

6.2 Limitations of the System

1. Payment Gateway Direct Settlement: While the platform supports Polar.sh for SaaS subscriptions and records internal cash/card settlement statuses, direct consumer online payment settlement (e.g., UPI, Razorpay) directly within the customer QR checkout flow remains a future enhancement.

2. Inventory & Stock Tracking: The platform tracks menu item availability via manual toggle flags but does not currently incorporate raw ingredient stock deduction or supplier inventory management.

3. Multi-Branch Centralized Administration: RosterPro supports multi-tenant organizations independently but lacks a consolidated parent-chain dashboard for managing multi-location restaurant franchises under a single umbrella.

6.3 Future Scope of the Project

• Direct Customer Payment Gateway Integration: Integrating native Razorpay and UPI intent flows into the mobile customer ordering flow for automated pre-paid order settlement.

• Raw Ingredient Inventory & Stock Depletion Engine: Implementing automated recipe-based ingredient deduction upon order completion to provide real-time inventory tracking and low-stock alerts.

• Multi-Location Franchise Hierarchy: Enabling multi-branch restaurant chains to manage unified brand catalogs while maintaining branch-specific pricing, staff permissions, and aggregated analytics.

• AI-Powered Demand Forecasting: Leveraging machine learning models over historical sales data to forecast daily ingredient requirements and suggest optimal dynamic pricing strategies.

CHAPTER 7
REFERENCES

[1] Hono Framework Documentation, "Fast, Lightweight, Web-standards based Web Framework," https://hono.dev/ (accessed Sep. 2026).

[2] Drizzle Team, "Drizzle ORM — TypeScript ORM that lets you write SQL with full type safety," https://orm.drizzle.team/ (accessed Sep. 2026).

[3] Next.js Documentation, "Next.js 16 App Router Architecture and Server Components," Vercel Inc., https://nextjs.org/docs (accessed Sep. 2026).

[4] Better Auth Documentation, "Comprehensive TypeScript Authentication Framework for Web Applications," https://better-auth.com/ (accessed Sep. 2026).

[5] Upstash Documentation, "Serverless Redis and Realtime Pub/Sub Documentation," https://docs.upstash.com/ (accessed Sep. 2026).

[6] National Institute of Standards and Technology (NIST), "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM)," NIST Special Publication 800-38D, 2007.

[7] PostgreSQL Global Development Group, "PostgreSQL 16 Database System Documentation," https://www.postgresql.org/docs/16/ (accessed Sep. 2026).

[8] Bun Runtime Documentation, "Bun: Fast all-in-one JavaScript runtime & toolkit," https://bun.sh/docs (accessed Sep. 2026).

[9] Turborepo Documentation, "High-performance build system for TypeScript monorepos," Vercel Inc., https://turbo.build/repo/docs (accessed Sep. 2026).

[10] Zod Library Documentation, "TypeScript-first schema validation with static type inference," https://zod.dev/ (accessed Sep. 2026).

[11] Goods and Services Tax Council of India, "GST Rate Schedule for Restaurant and Food Services," Central Board of Indirect Taxes and Customs (CBIC), Government of India.

[12] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

[13] Hoffer, J. A., George, J. F., & Valacich, J. S. (2002). Modern Systems Analysis and Design (3rd ed.). Pearson Education.

GLOSSARY

AES-256-GCM	Advanced Encryption Standard in Galois/Counter Mode providing confidentiality and integrity.
API	Application Programming Interface for client-server networked communication.
B2B SaaS	Business-to-Business Software as a Service delivery model.

DFD / ERD	Data Flow Diagram and Entity-Relationship Diagram for functional and data modeling.
GST	Goods and Services Tax; statutory indirect taxation levied on restaurant food services.
KDS / KOT	Kitchen Display System and Kitchen Order Ticket for meal preparation tracking.
SSE	Server-Sent Events; unidirectional HTTP push streaming for real-time order updates.